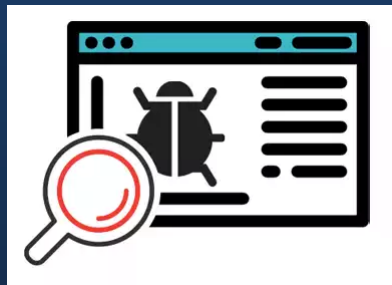


Testy jednostkowe - wprowadzenie



„[] opanowanie teorii jest zaledwie niezbędnym wstępem”

Grzegorz Szymkowiak

Pojęcie test jednostkowy

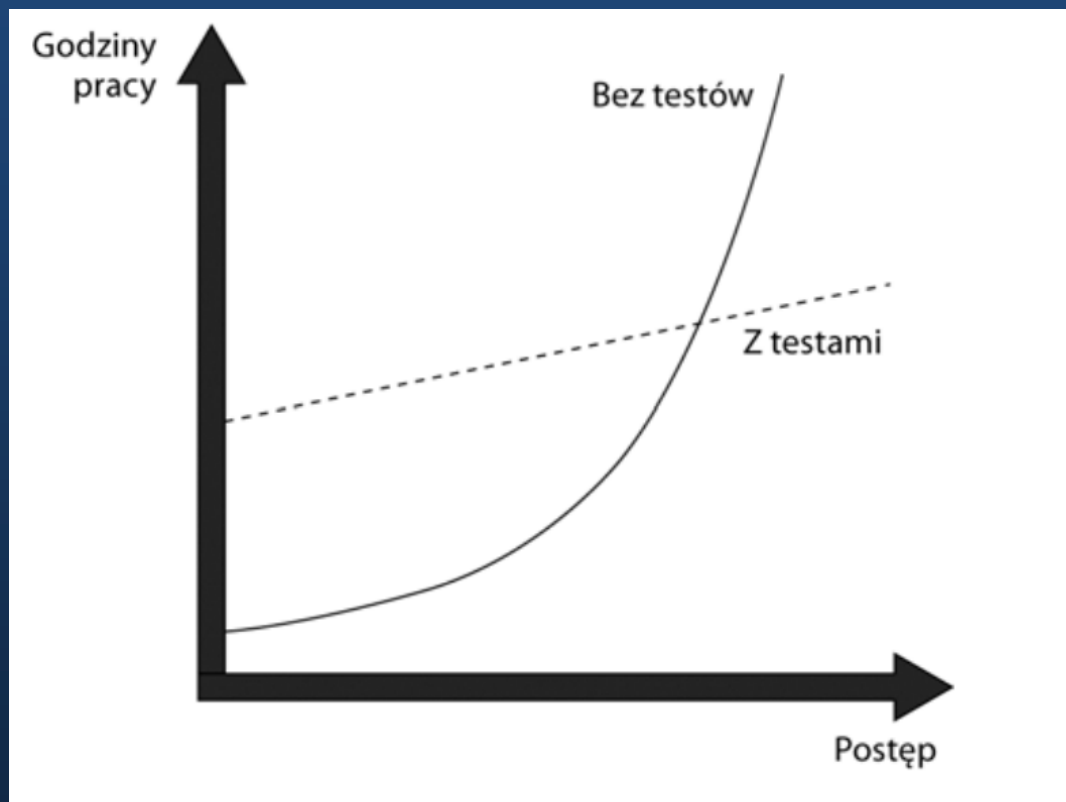
Test jednostkowy

- **Test jednostkowy** to rodzaj testu, który sprawdza jednostkę aplikacji (najczęściej funkcja, metoda, rzadziej klasa) w izolacji od innych fragmentów kodu, w celu zweryfikowania poprawnego działania

Test jednostkowy

- Projekt pozbawiony testów jednostkowych rozpoczyna się wysokim tempem pracy, ale szybko zwalnia (zjawisko to znane jest jako **entropia oprogramowania**)

Cel testu jednostkowego



Test jednostkowy

- *„Dyskusja o tym, czy powinno się je przeprowadzać, została dawno i bezapelacyjnie rozstrzygnięta. O ile nie pracujesz nad projektem zabawką, werdykt brzmi: tak, powinno.”*

Vladimir Khorikov

Test jednostkowy

- *„Proporcja między kodem produkcyjnym a kodem testowym może wynosić od 1:1 do nawet 1:3 (każdej linii kodu produkcyjnego odpowiada od jednej do trzech linii kodu testów)”*

Vladimir Khorikov

Cechy testu jednostkowego

- zasada FIRST

Cechy testu jednostkowego

- Akronim **FIRST** opisuje cechy dobrych testów jednostkowych:
 - **F (Fast)** – testy powinny działać szybko
 - **I (Independent)** – testy powinny być niezależne od siebie
 - **R (Repeatable)** – wynik testu powinien być zawsze taki sam
 - **S (Self-validating)** – testy powinny same wskazywać, czy są zaliczone (PASS) czy nie (FAIL)
 - **T (Timely)** – testy powinny być pisane przed lub równocześnie z kodem (TDD)

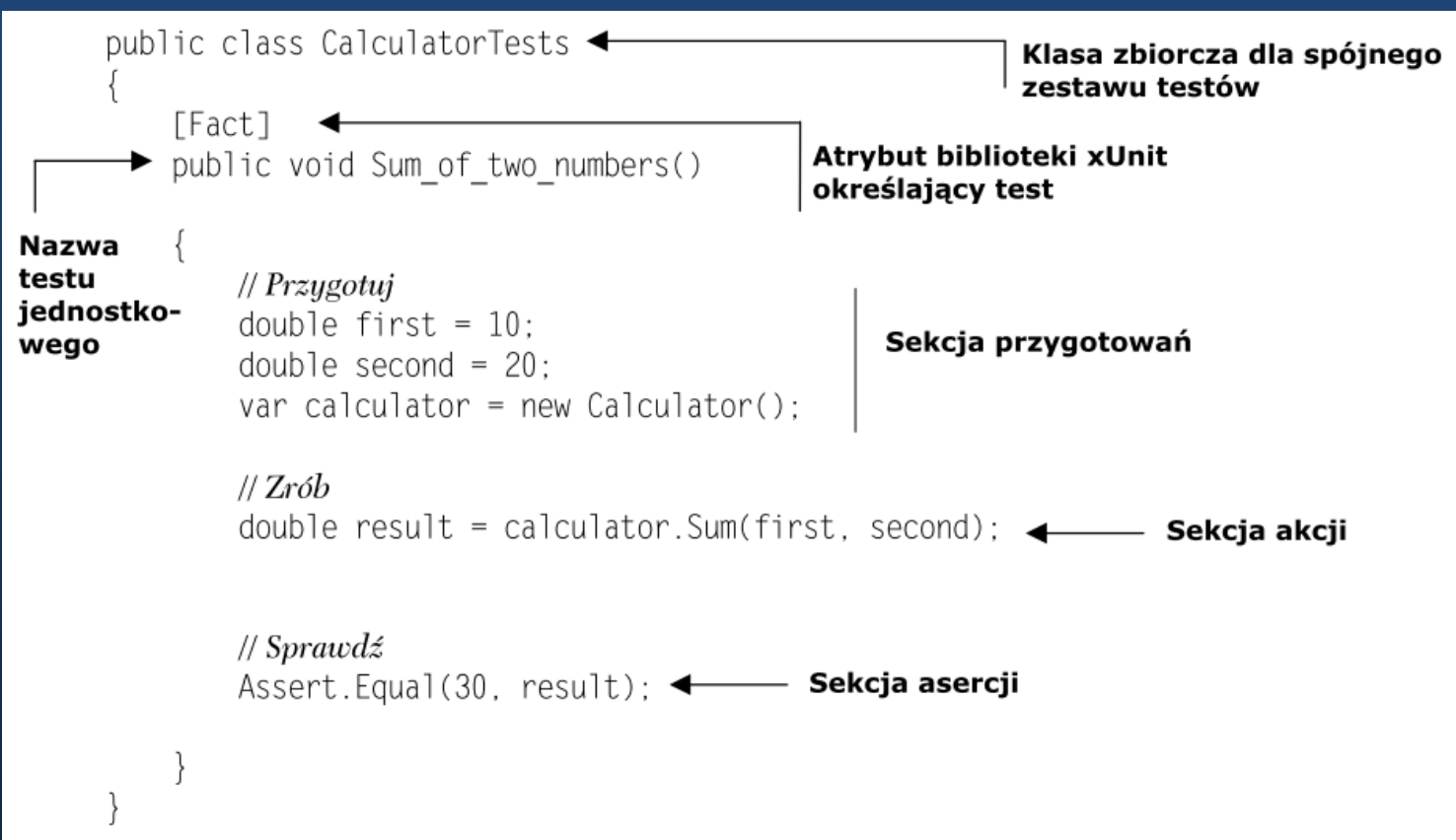
Struktura testu jednostkowego

zasada AAA (Arrange-Act-Assert)

Struktura testu jednostkowego

- Testy jednostkowe buduje się, wykorzystując **wzorzec AAA**, który dzieli test na trzy części:
 - **Arrange** (sekcja przygotowań) – ustawienie warunków początkowych
 - **Act** (sekcja wykonania) – uruchomienie testowego fragmentu kodu
 - **Assert** (sekcja asercji) – weryfikacja, czy wynik jest zgodny z oczekiwaniami

Struktura testu jednostkowego



Frameworki testów jednostkowych

Frameworki testów jednostkowych

- Frameworki testów jednostkowych są dostępne dla różnych języków programowania – frameworki te automatyzują proces testowy oraz pomagają w utrzymaniu wysokiej jakości kodu

Frameworki testów jednostkowych

- Dla języka **JavaScript/Node.js** to:
 - **Jest** – framework rozwijany przez Facebook, popularny w projektach React czy ogólnie JS
 - **Mocha** – elastyczny framework, często używany z Chai (biblioteka asercji)
 - **Jasmine** – framework, który nie wymaga dodatkowych bibliotek do asercji

Frameworki testów jednostkowych

- Dla języka C# to:
 - **MSTest** – zintegrowany z Visual Studio
 - **NUnit** – popularny framework inspirowany JUnit
 - **xUnit** – nowoczesny framework, uznawany za następcę NUnit

Frameworki testów jednostkowych

- Dla C++ to:
 - **Microsoft Native Unit Test** – wbudowany w Visual Studio
 - **Google Test (gtest)** – jeden z najczęściej używanych frameworków do testów w C++
 - **Catch2** – prosty w użyciu framework dla C++

Frameworki testów jednostkowych

- Główne funkcje frameworków testów jednostkowych:
 - automatycznie uruchamiają i raportują wyniki
 - sprawdzają, czy wyniki są zgodne z oczekiwaniami
 - tworzą przejrzyste raporty (PASS/FAIL) z informacjami o błędach

Frameworki testów jednostkowych

- Główne funkcje frameworków testów jednostkowych:
 - dają możliwość użycia mocków (symulują zachowanie zależności) i stubów (gotowe odpowiedzi do wywołania) do symulacji zależności
 - wykorzystują wzorzec:
 - **AAA** (Arrange, Act, Assert)
 - **GDD** (Given, When, Then)

Czego nie obejmują testy
jednostkowe?

Czego nie obejmują testy?

- Interakcji między wieloma modułami - to testy integracyjne
- Komunikacji z bazą danych lub API - zamiast tego używa się mocków (symulacja zewnętrznego API)
- Interfejsu użytkownika - to testy end-to-end lub UI
- Zachowań systemowych w skali całej aplikacji - to testy systemowe

www.szymkowiak.tychy.pl

KONIEC

Grzegorz Szymkowiak